Quality per Joule: A Comparative Study of Energy Efficiency in Autoencoders, VAEs, GANs, WGAN-GPs, and Diffusion Models

---

Md. Masud Mazumder

ECE 7650 – Deep Generative Modeling: Theory & Application

Winter 2026

GitHub Repository: **Link**

Date: April 2, 2026

I, **Md. Masud Mazumder**, attest that the work I am submitting is my own work and that it has not been copied/plagiarized from online or other sources. Any sourced material used for completing this work has been properly cited.

Md. Masud Mazumder

Acknowledgment of AI Assistance

ChatGPT were used to assist in organizing the structure of this document and reviewing relevant background concepts related to generative models and energy measurement. Code conversion from the provided notebook to PyTorch was initially done using ChatGPT and later inspected and fixed by the author. In some cases, text written by the author was polished further by ChatGPT for better presentation and enhanced readability. All ideas, experimental design decisions, implementation, and final written content reflect the author's own understanding and academic work.

Contents

1 Learning Outcomes **4**

2 Problem Statement **4**

3 Background Theory **4**

4 Methodology **5**

5 Results and Discussion **7**

6 Summary **12**

7 Key Code Snippets **12**

7.1 Autoencoder 12

7.1.1 Encoder 12

7.1.2 Decoder 13

7.2 Variational Autoencoder 13

7.2.1 Encoder 13

7.2.2 Decoder 14

7.3 DCGAN 14

7.3.1 Generator 15

7.3.2 Discriminator 15

7.4 WGAN-GP 16

7.4.1 Generator 16

7.4.2 Critic (Discriminator) 17

7.5 DDPM 18

7.5.1 Sinusoidal Timestep Embedding 18

7.5.2 MLP for Time Embedding 18

7.5.3 Residual Block 18

7.5.4 Attention Block 19

7.5.5 U-Net for DDPM 20

7.5.6 Noise Scheduler for DDPM 23

7.6 Energy-Tracked Training Loop (Autoencoder) 24

7.7 FID Computation 26

7.8 WGAN-GP Gradient Penalty 27

7.9 DDPM Noise Prediction Training Step 27

1 Learning Outcomes

From this project, one can learn the following:

- Measure **GPU energy consumption** during model training using CodeCarbon and NVIDIA's NVML interface.
- Compute and interpret **Fréchet Inception Distance (FID)** as a quantitative measure of generative quality.
- **Implement and train** representative lightweight generative architectures for 32×32 face image synthesis on CelebA.
- Propose and apply a unified metric–**Quality per Joule (QpJ)**–to jointly evaluate generative quality and computational energy cost.
- Analyze the **trade-off between generative quality and energy consumption**.
- Discuss the implications for **sustainable and energy-aware generative AI**.
- Understand and compare the **objective functions and training dynamics** of Autoencoders, VAEs, DCGANs, WGAN-GPs, and DDPMs.

2 Problem Statement

Generative models such as Autoencoders (AEs), Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), Wasserstein GANs with Gradient Penalty (WGAN-GPs), and Denoising Diffusion Probabilistic Models (DDPMs) have achieved impressive performance in image synthesis, but their computational and energy costs vary significantly due to differences in objective functions, optimization dynamics, and sampling procedures. While generative quality is commonly evaluated using metrics such as FID, energy efficiency is rarely incorporated into comparative analysis. This project investigates the energy–quality trade-off of these five model families under controlled experimental conditions–identical dataset (CelebA, 32×32), same batch size (64), same number of epochs (30), and same hardware (NVIDIA RTX 2080 SUPER)–and proposes a unified metric, **Quality per Joule (QpJ)**, defined as $\mathbf{QpJ} = 1/(\mathbf{FID} \times \mathbf{Energy})$, to quantify generative efficiency.

3 Background Theory

This project compares five major families of deep generative models. **Autoencoders** [1] learn a compressed latent representation $\mathbf{z} = f_\theta(\mathbf{x})$ and reconstruct the input via $\hat{\mathbf{x}} = g_\phi(\mathbf{z})$, trained by minimizing $\mathscr{L}_{\text{AE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$. They are stable and efficient but lack a structured latent space for generation. **Variational Autoencoders** [2] extend AEs by imposing a Gaussian posterior

$q_\phi(\mathbf{z}|\mathbf{x})$ and training with the ELBO: $\mathscr{L}_{\text{VAE}} = \mathbb{E}[\|\mathbf{x}-\hat{\mathbf{x}}\|^2] + \beta \cdot D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$, enabling sampling via $\mathbf{z} \sim \mathscr{N}(\mathbf{0},\mathbf{I})$ but often producing blurrier outputs.

DCGANs [3, 4] train a generator G and discriminator D adversarially: $\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1-D(G(\mathbf{z})))]$, producing sharp images but risking instability. **WGAN-GP** [5] replaces the JS divergence with the Wasserstein distance and enforces a Lipschitz constraint via gradient penalty: $\mathscr{L}_C = \mathbb{E}[C(G(\mathbf{z}))] - \mathbb{E}[C(\mathbf{x})] + \lambda\, \mathbb{E}[(\|\nabla_{\hat{\mathbf{x}}} C(\hat{\mathbf{x}})\|_2 - 1)^2]$, improving stability but increasing compute through multi-step critic updates. **DDPMs** [6] generate samples by iteratively denoising from $\mathbf{x}_T \sim \mathscr{N}(\mathbf{0},\mathbf{I})$, training a U-Net ε_θ to predict noise: $\mathscr{L}_{\text{DDPM}} = \mathbb{E}[\|\varepsilon - \varepsilon_\theta(\mathbf{x}_t, t)\|^2]$. They achieve high quality but are computationally expensive due to the T-step iterative process.

FID [7] measures the distance between real and generated Inception-v3 feature distributions: $\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$; lower is better. **Energy** is measured as $E = P_{\text{avg}} \times t$ using CodeCarbon [11] with NVML. The **Quality per Joule** metric $\text{QpJ} = 1/(\text{FID} \times E)$ rewards models achieving low FID with minimal energy. As AI scales [9, 10], such energy-aware metrics become essential for sustainable model selection.

4 Methodology

Figure 1 outlines the end-to-end experimental pipeline adopted in this study, spanning seven stages: data preparation, preprocessing, model training with simultaneous energy tracking, per-model output collection, FID-based evaluation, QpJ metric computation, and comparative analysis. All these stages are summarized below.

Dataset. CelebA [8] ($\sim$162K training / $\sim$20K validation) is preprocessed by center-cropping to 178×178, resizing to 32×32, and normalizing to $[-1, 1]$. Figure 2 shows a sample batch.

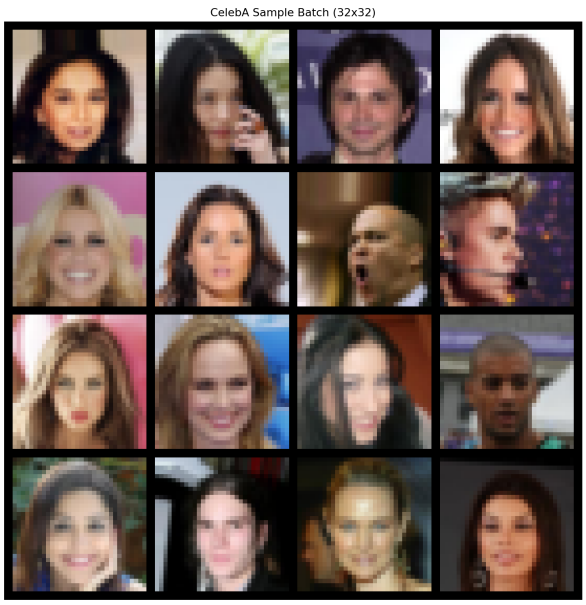

Figure 2: CelebA sample batch after preprocessing (32×32).

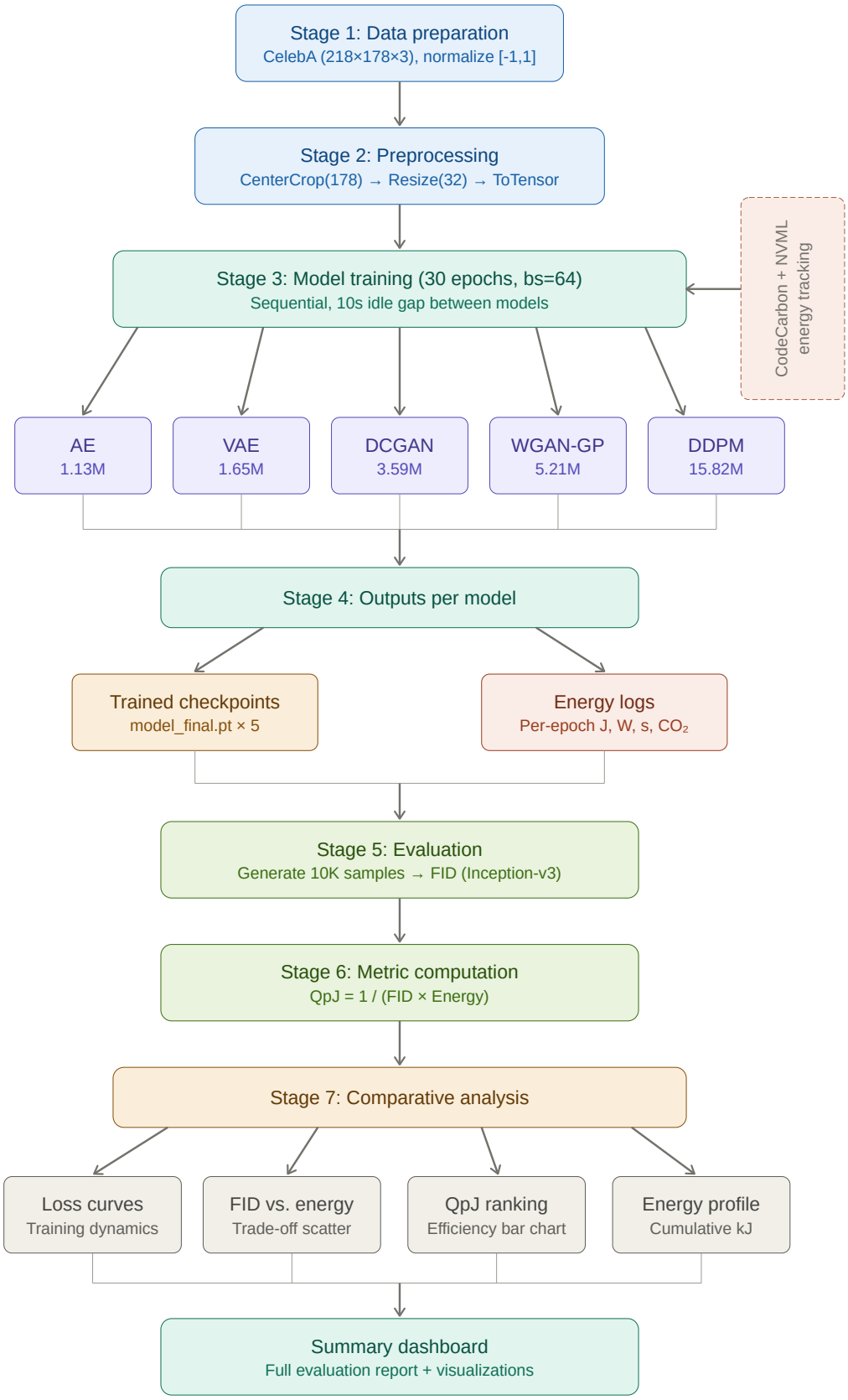

Figure 1: Quality per Joule – full experimental pipeline

Models. Five convolutional architectures for 32×32 images are implemented: (1) a *Convolutional AE* with a 5-block encoder/decoder and a 128-dim latent space (1.13M params); (2) a *VAE* with an identical backbone plus $\mu/\log\sigma^2$ heads and KL (1.65M params); (3) a *DCGAN* with a 6-block generator/discriminator and label smoothing (3.59M params); (4) a *WGAN-GP* sharing the DCGAN generator, with a BatchNorm-free critic, $\lambda_{\text{GP}} = 10$, $n_{\text{critic}} = 3$ (5.21M params); and (5) a *DDPM* with a 5-level U-Net, sinusoidal time embeddings, and self-attention, $T{=}1000$ (15.82M params).

Training. All models are trained for 30 epochs, batch size 64, learning rate 2×10^{-4} (Adam), and seed 42, on a single RTX 2080 SUPER. A 10-second idle gap is inserted between training stages for clean energy separation. Per-epoch energy is logged via NVML, and total emissions are tracked via CodeCarbon.

Evaluation. FID is computed using `torchmetrics` (Inception-v3, 2048-dim features) on 10,000 generated samples vs. 10,000 fixed validation images. For DDPM, $T{=}1000$ sampling steps are used during evaluation. QpJ is computed as $1/(\text{FID} \times \text{Energy in J})$.

Table 1 summarizes the training configuration.

Table 1: Training hyperparameters for each model.

Parameter	AE	VAE	DCGAN	WGAN-GP	DDPM
Latent dim	128	128	128	128	–
Epochs / Batch	30 / 64	30 / 64	30 / 64	30 / 64	30 / 64
LR / Optimizer	2e-4 / Adam	2e-4 / Adam	2e-4 / Adam	2e-4 / Adam	2e-4 / Adam
β_1/β_2	–	–	0.5/0.999	0.5/0.9	0.0/0.999
Special	–	–	Smooth 0.9	$\lambda{=}10$, $n_c{=}3$	$T{=}1000$

Hardware: Intel i7-9700K, NVIDIA RTX 2080 SUPER (8 GB), 31 GB RAM, Ubuntu Linux, Python 3.10, PyTorch, CodeCarbon 3.2.5.

5 Results and Discussion

Table 2 summarizes the key evaluation metrics.

Table 2: Main results: FID, energy, training time, parameters, and Quality per Joule.

Model	FID ↓	Energy (kJ)	Time (min)	Params	QpJ $\times 10^{-7}$
Autoencoder	33.45	103.9	12.2	1.13M	**2.900**
VAE	117.81	108.3	12.7	1.65M	0.800
DCGAN	**15.20**	1,115.1	77.6	3.59M	0.600
WGAN-GP	20.52	4,466.2	320.6	5.21M	0.100
DDPM	20.30	1,098.0	84.5	15.82M	0.400

Generative quality. DCGAN achieves the best FID (15.20), followed by DDPM (20.30) and WGAN-GP (20.52). The Autoencoder reaches a moderate 33.45, while the VAE's high FID (117.81) reflects inherent blurriness from KL regularization. Figures 3–7 confirm these trends visually–DCGAN and WGAN-GP produce the sharpest faces, DDPM shows good diversity, and the VAE is notably blurry.

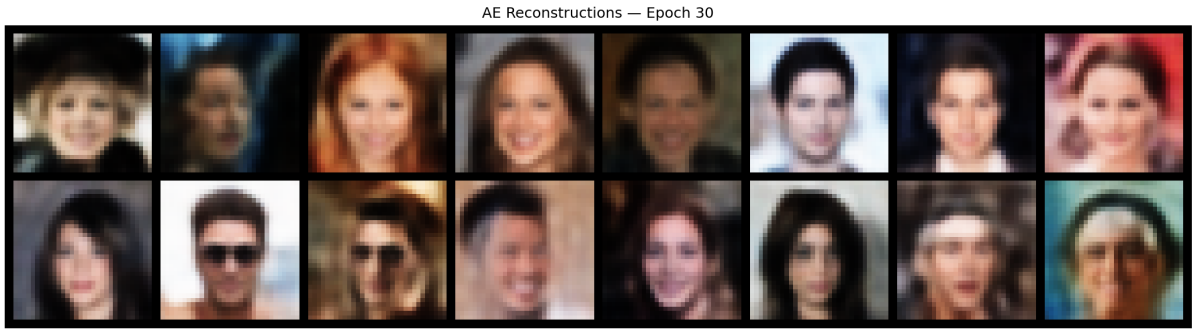

Figure 3: Autoencoder – Reconstructions (Epoch 30).

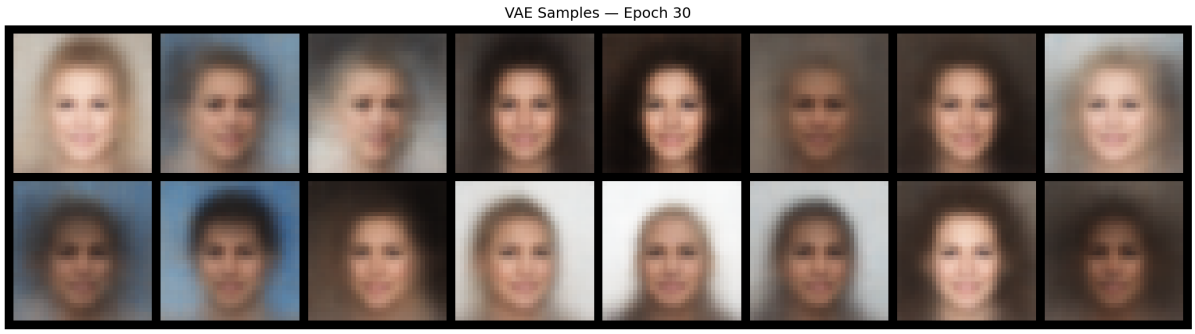

Figure 4: VAE – Generated Samples (Epoch 30).

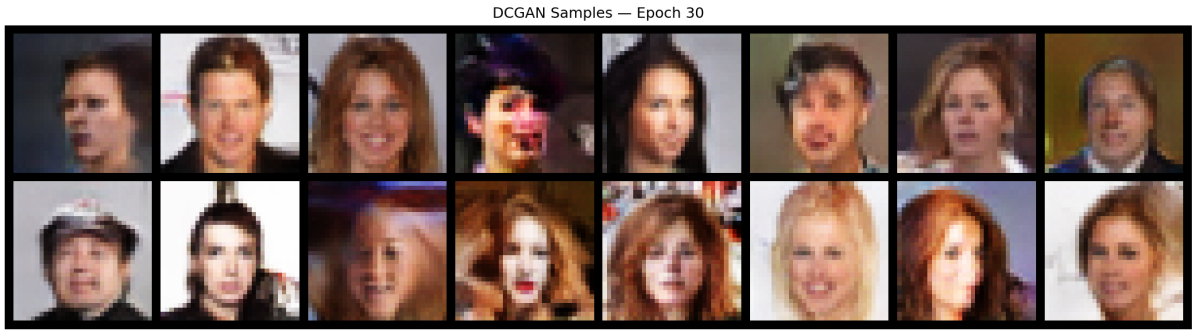

Figure 5: DCGAN – Generated Samples (Epoch 30).

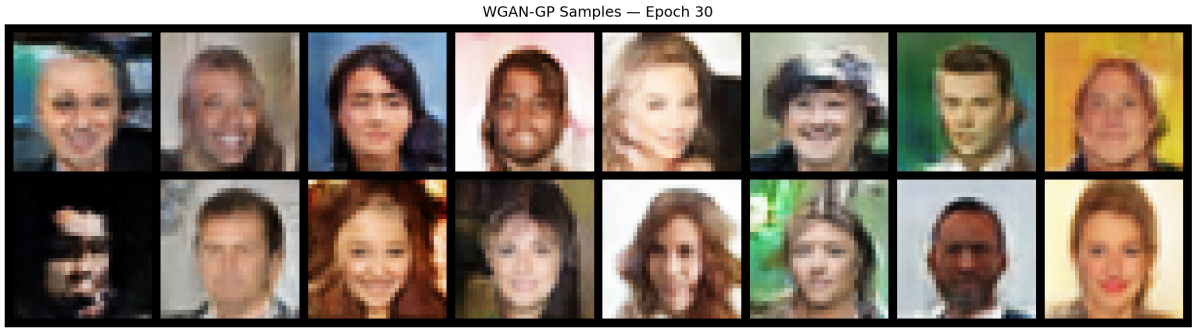

Figure 6: WGAN-GP – Generated Samples (Epoch 30).

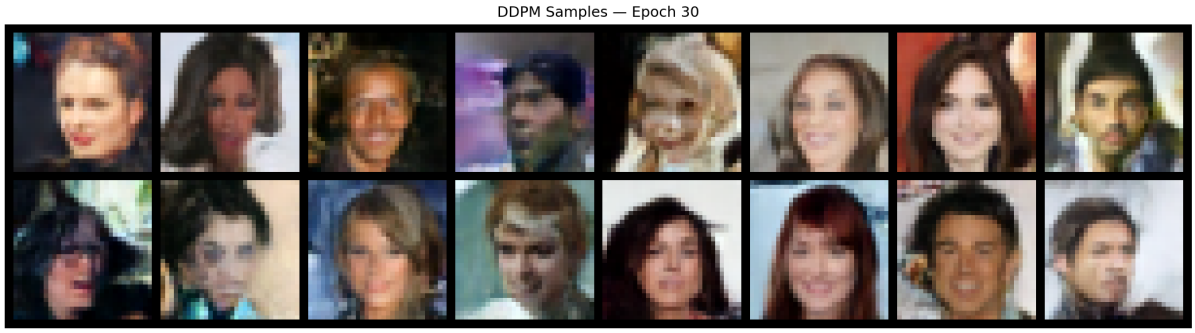

Figure 7: DDPM – Generated Samples (Epoch 30).

Energy consumption. As we can see in Table 2, AE and VAE consume ∼104–108 kJ at ∼113 W, while DCGAN and DDPM require ∼1,100 kJ at ∼220 W. WGAN-GP consumes 4,466 kJ–**42× the Autoencoder**–due to its 3:1 critic update ratio and per-sample gradient penalty. CO_2 emissions range from 0.15 g (AE) to 6.54 g (WGAN-GP), a 43× difference.

Quality per Joule. The Autoencoder dominates with QpJ = 2.9×10^{-7} (3.6× higher than VAE, 4.8× higher than DCGAN). WGAN-GP ranks last (0.1×10^{-7}): it achieves only marginally better FID than DDPM (20.52 vs. 20.30) at 4× the energy. The energy–quality trade-off is highly non-linear–diminishing returns are severe.

Key insight. No single model dominates both dimensions. Quality-first → DCGAN. Efficiency-first → AE. Balanced → DDPM (near-DCGAN quality, comparable energy, no mode collapse). WGAN-GP's energy overhead is not justified by its quality in this setting.

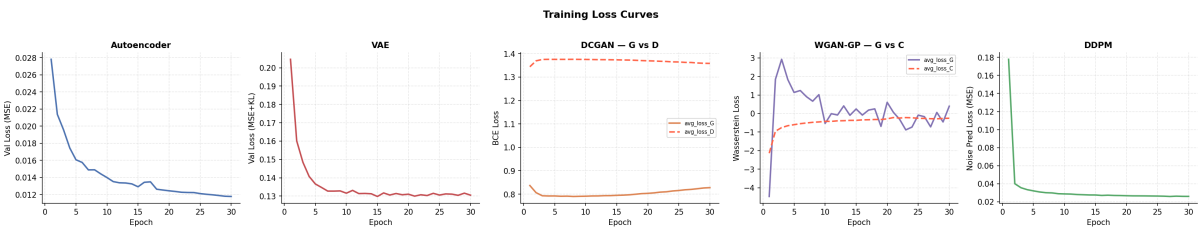

Figure 8: Training loss curves for all five models over 30 epochs.

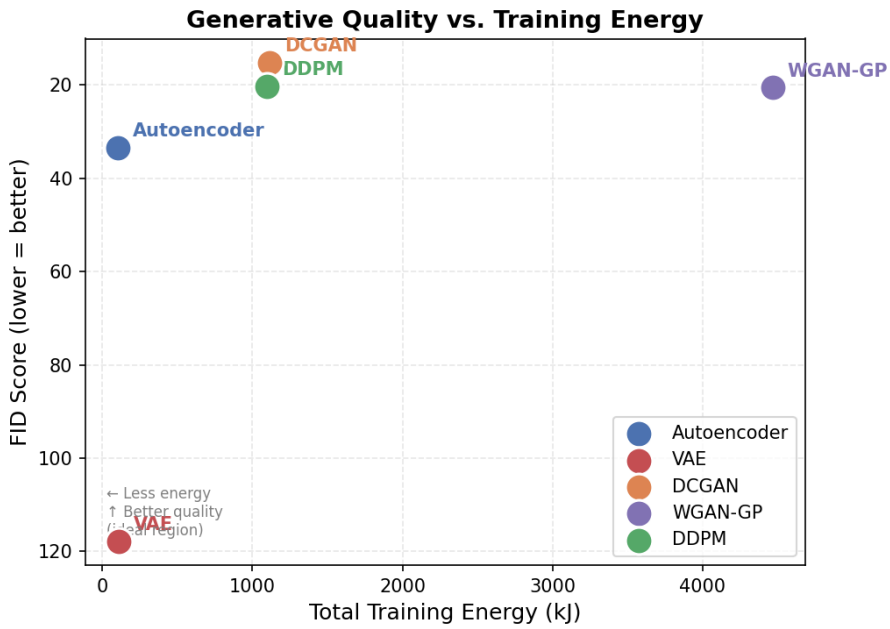

Figure 9: FID vs. total training energy. Ideal region: upper-left (low energy, low FID).

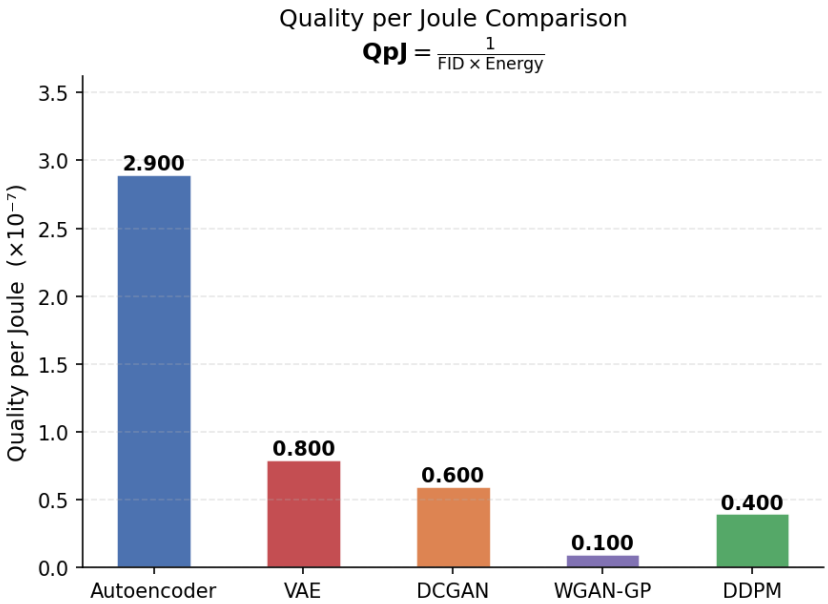

Figure 10: Quality per Joule comparison ($\times 10^{-7}$; higher is better).

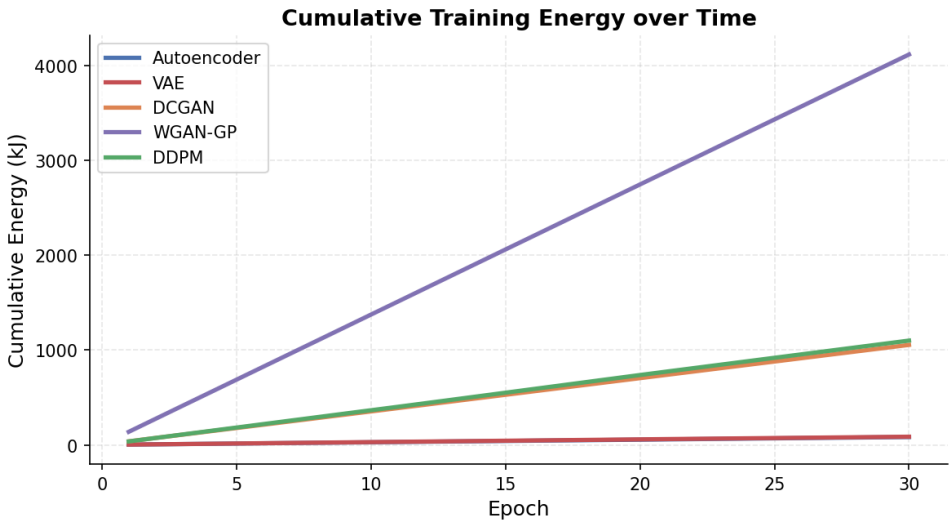

Figure 11: Cumulative training energy over 30 epochs.

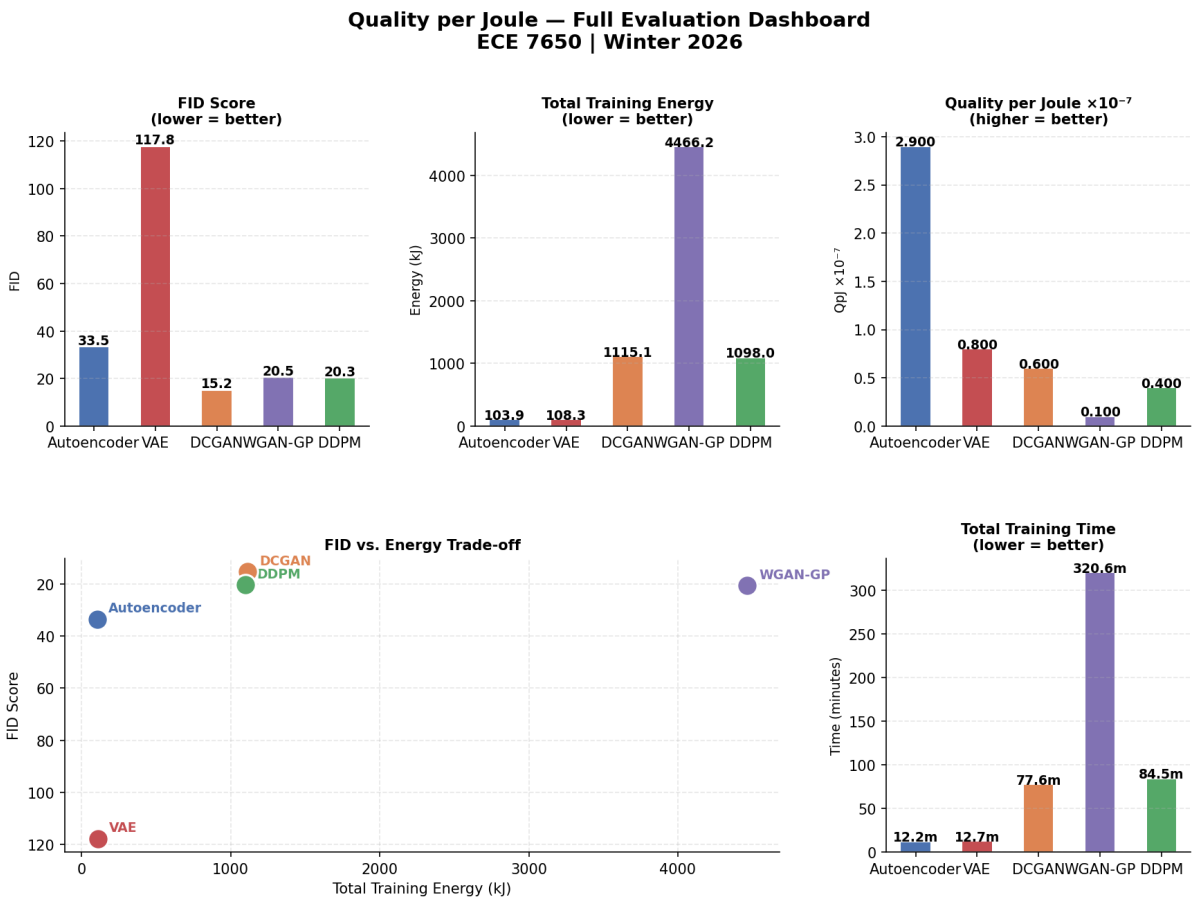

Figure 12: Full evaluation dashboard: FID, energy, QpJ, FID–energy trade-off, training time.

6 Summary

This project presents a controlled comparative study of energy efficiency across five deep generative model families–Autoencoder, VAE, DCGAN, WGAN-GP, and DDPM–trained on CelebA at 32×32 resolution under identical conditions. DCGAN achieves the best generative quality (FID = 15.20), while the Autoencoder attains the highest Quality per Joule (2.9×10^{-7}) due to its minimal energy footprint (103.9 kJ). WGAN-GP incurs the highest energy cost (4,466 kJ, 42× the Autoencoder) without commensurate quality gains over DCGAN. DDPM offers a compelling balance of quality (FID = 20.30), stability, and energy efficiency. The proposed Quality per Joule metric provides a practical tool for incorporating energy awareness into generative model selection, and the results underscore that energy–quality trade-offs are highly non-linear, with severe diminishing returns at the high-energy end of the spectrum.

7 Key Code Snippets

The following code snippets illustrate the core implementation. Full source code is available in the GitHub repository (link).

7.1 Autoencoder

7.1.1 Encoder

```
class Encoder(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2,
               padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, stride=2,
               padding=1),
            nn.ReLU(inplace=True),
        )
        self.fc = nn.Linear(64 * 8 * 8, latent_dim)
    def forward(self, x):
        x = self.conv_layers(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

7.1.2 Decoder

```python
class Decoder(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.fc = nn.Sequential(
            nn.Linear(latent_dim, 64 * 8 * 8),
            nn.ReLU(inplace=True),
        )
        self.deconv_layers = nn.Sequential(
            nn.ConvTranspose2d(64, 64, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(32, 3, kernel_size=3, stride=1, padding=1),
            nn.Tanh(),
        )

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        x = self.fc(z)
        x = x.view(x.size(0), 64, 8, 8)
        x = self.deconv_layers(x)
        return x
```

7.2 Variational Autoencoder

7.2.1 Encoder

```python
class VAEEncoder(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
```

```
            nn.ReLU(inplace=True),
        )

        self.fc_mu     = nn.Linear(64 * 8 * 8, latent_dim)
        self.fc_logvar = nn.Linear(64 * 8 * 8, latent_dim)

    def forward(self, x: torch.Tensor):
        h = self.conv_layers(x)
        h = h.view(h.size(0), -1)   # (B, 4096)
        return self.fc_mu(h), self.fc_logvar(h)
```

7.2.2 Decoder

```
class VAEDecoder(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.fc = nn.Sequential(
            nn.Linear(latent_dim, 64 * 8 * 8),
            nn.ReLU(inplace=True),
        )

        self.deconv_layers = nn.Sequential(
            nn.ConvTranspose2d(64, 64, kernel_size=3, stride=2,
               padding=1, output_padding=1),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2,
               padding=1, output_padding=1),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(32, 3, kernel_size=3, stride=1,
               padding=1),
            nn.Tanh(),
        )

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        x = self.fc(z)
        x = x.view(x.size(0), 64, 8, 8)
        return self.deconv_layers(x)
```

7.3 DCGAN

7.3.1 Generator

```python
class Generator(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.fc = nn.Sequential(
            nn.Linear(latent_dim, 4 * 4 * 128),
        )
        self.net = nn.Sequential(
            nn.ConvTranspose2d(128, 128, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.ConvTranspose2d(128, 256, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.ConvTranspose2d(256, 512, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(512, 3, kernel_size=5, stride=1, padding=2),
            nn.Tanh(),
        )

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        x = self.fc(z)
        x = x.view(x.size(0), 128, 4, 4)
        return self.net(x)
```

7.3.2 Discriminator

```python
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

```

```
            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(128, 128, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(128 * 4 * 4, 1),
            nn.Sigmoid(),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x).view(-1)
```

7.4 WGAN-GP

7.4.1 Generator

```
class Generator(nn.Module):
    def __init__(self, latent_dim: int = 128):
        super().__init__()
        self.latent_dim = latent_dim
        self.fc = nn.Sequential(
            nn.Linear(latent_dim, 4 * 4 * 256, bias=False),
            nn.BatchNorm1d(4 * 4 * 256),
            nn.LeakyReLU(0.2, inplace=True),
        )
        self.net = nn.Sequential(
            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(256, 128, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(128, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
```

```
            nn.LeakyReLU(0.2, inplace=True),

            nn.Upsample(scale_factor=2, mode='nearest'),
            nn.Conv2d(64, 3, kernel_size=3, stride=1,
                padding=1, bias=False),
            nn.BatchNorm2d(3),
            nn.Tanh(),
        )

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        x = self.fc(z)
        x = x.view(x.size(0), 256, 4, 4)
        return self.net(x)
```

7.4.2 Critic (Discriminator)

```
class Critic(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=5, stride=2,
                padding=2, bias=True),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(64, 128, kernel_size=5, stride=2,
                padding=2, bias=True),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Dropout(0.3),

            nn.Conv2d(128, 256, kernel_size=5, stride=2,
                padding=2, bias=True),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Dropout(0.3),

            nn.Conv2d(256, 512, kernel_size=5, stride=2,
                padding=2, bias=True),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Flatten(),
            nn.Dropout(0.2),
            nn.Linear(512 * 2 * 2, 1),
```

```
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x).view(-1)
```

7.5 DDPM

7.5.1 Sinusoidal Timestep Embedding

```
class SinusoidalTimeEmbedding(nn.Module):
    def __init__(self, dim: int):
        super().__init__()
        self.dim = dim

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        half = self.dim // 2
        freqs = torch.exp(
            -math.log(10000) * torch.arange(half,
               device=t.device).float() / (half - 1)
        )
        args = t[:, None].float() * freqs[None]
        return torch.cat([torch.sin(args), torch.cos(args)],
           dim=-1)
```

7.5.2 MLP for Time Embedding

```
class TimeMLP(nn.Module):
    def __init__(self, dim: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim, dim),
            nn.SiLU(),
            nn.Linear(dim, dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

7.5.3 Residual Block

```
class ResidualBlock(nn.Module):
```

```
    def __init__(self, in_ch: int, out_ch: int, time_dim: int, groups: int = 8):
        super().__init__()
        self.norm1 = nn.GroupNorm(groups, in_ch)
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, padding=1)
        self.time_proj = nn.Linear(time_dim, out_ch)
        self.norm2 = nn.GroupNorm(groups, out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1)
        self.res_conv = nn.Conv2d(in_ch, out_ch, 1) if in_ch != out_ch else nn.Identity()

    def forward(self, x: torch.Tensor, t_emb: torch.Tensor) -> torch.Tensor:
        h = self.conv1(F.silu(self.norm1(x)))
        h = h + self.time_proj(F.silu(t_emb))[:, :, None, None]
        h = self.conv2(F.silu(self.norm2(h)))
        return h + self.res_conv(x)
```

7.5.4 Attention Block

```
class AttentionBlock(nn.Module):
    def __init__(self, ch: int, groups: int = 8):
        super().__init__()
        self.norm = nn.GroupNorm(groups, ch)
        self.q = nn.Linear(ch, ch)
        self.k = nn.Linear(ch, ch)
        self.v = nn.Linear(ch, ch)
        self.proj = nn.Linear(ch, ch)
        self.scale = ch ** -0.5

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, C, H, W = x.shape
        h = self.norm(x).view(B, C, H * W).permute(0, 2, 1)  # (B, HW, C)
        q, k, v = self.q(h), self.k(h), self.v(h)
        attn = torch.bmm(q, k.transpose(1, 2)) * self.scale
        attn = attn.softmax(dim=-1)
        out = torch.bmm(attn, v)
        out = self.proj(out)
        return x + out.permute(0, 2, 1).view(B, C, H, W)
```

7.5.5 U-Net for DDPM

```python
class UNet(nn.Module):
    def __init__(
        self,
        in_ch: int = 3,
        base_ch: int = 32,
        channel_multiplier: list = None,
        has_attention: list = None,
        num_res_blocks: int = 2,
        norm_groups: int = 8,
        time_dim: int = None,
    ):
        super().__init__()
        if channel_multiplier is None:
            channel_multiplier = [1, 2, 4, 8]
        if has_attention is None:
            has_attention = [False, False, True, True]
        if time_dim is None:
            time_dim = base_ch * 4

        widths = [base_ch * m for m in channel_multiplier]
        num_levels = len(widths)

        self.time_emb = nn.Sequential(
            SinusoidalTimeEmbedding(time_dim),
            TimeMLP(time_dim),
        )

        self.init_conv = nn.Conv2d(in_ch, base_ch, 3, padding=1)

        # Encoder (DownBlocks)
        self.down_blocks = nn.ModuleList()
        self.down_samples = nn.ModuleList()
        ch = base_ch
        skip_channels = [base_ch]

        for i in range(num_levels):
            block_list = nn.ModuleList()
            attn_list = nn.ModuleList()
            for _ in range(num_res_blocks):
```

```
                block_list.append(ResidualBlock(ch, widths[i], time_dim, norm_groups))
                ch = widths[i]
                if has_attention[i]:
                    attn_list.append(AttentionBlock(ch, norm_groups))
                else:
                    attn_list.append(nn.Identity())
                skip_channels.append(ch)
            self.down_blocks.append(nn.ModuleDict({
                'res': block_list, 'attn': attn_list
            }))
            if i < num_levels - 1:
                self.down_samples.append(nn.Conv2d(ch, ch, 3, stride=2, padding=1))
                skip_channels.append(ch)
            else:
                self.down_samples.append(None)

        # Middle Block
        self.mid_res1 = ResidualBlock(ch, ch, time_dim, norm_groups)
        self.mid_attn = AttentionBlock(ch, norm_groups)
        self.mid_res2 = ResidualBlock(ch, ch, time_dim, norm_groups)

        # Decoder (UpBlocks)
        self.up_blocks = nn.ModuleList()
        self.up_samples = nn.ModuleList()
        for i in reversed(range(num_levels)):
            block_list = nn.ModuleList()
            attn_list = nn.ModuleList()
            for _ in range(num_res_blocks + 1):
                skip_ch = skip_channels.pop()
                block_list.append(ResidualBlock(ch + skip_ch, widths[i], time_dim, norm_groups))
                ch = widths[i]
                if has_attention[i]:
                    attn_list.append(AttentionBlock(ch, norm_groups))
                else:
```

```
                    attn_list.append(nn.Identity())
            self.up_blocks.append(nn.ModuleDict({
                'res': block_list, 'attn': attn_list
            }))
            if i > 0:
                self.up_samples.append(nn.Sequential(
                    nn.Upsample(scale_factor=2, mode='nearest'),
                    nn.Conv2d(ch, ch, 3, padding=1),
                ))
            else:
                self.up_samples.append(None)

        # Output
        self.out_norm = nn.GroupNorm(norm_groups, ch)
        self.out_conv = nn.Conv2d(ch, in_ch, 3, padding=1)

    def forward(self, x: torch.Tensor, t: torch.Tensor) ->
        torch.Tensor:
        t_emb = self.time_emb(t)
        x = self.init_conv(x)
        skips = [x]

        # Encoder
        for i, (block, downsample) in
            enumerate(zip(self.down_blocks, self.down_samples)):
            for res, attn in zip(block['res'], block['attn']):
                x = res(x, t_emb)
                x = attn(x)
                skips.append(x)
            if downsample is not None:
                x = downsample(x)
                skips.append(x)

        # Middle
        x = self.mid_res1(x, t_emb)
        x = self.mid_attn(x)
        x = self.mid_res2(x, t_emb)

        # Decoder
        for block, upsample in zip(self.up_blocks,
            self.up_samples):
```

```
            for res, attn in zip(block['res'], block['attn']):
                x = torch.cat([x, skips.pop()], dim=1)
                x = res(x, t_emb)
                x = attn(x)
            if upsample is not None:
                x = upsample(x)

        return self.out_conv(F.silu(self.out_norm(x)))
```

7.5.6 Noise Scheduler for DDPM

```
class DDPMScheduler:
    """
    Notebook: GaussianDiffusion - linear beta schedule.
    """
    def __init__(self, T: int = 1000, beta_start: float = 1e-4,
        beta_end: float = 0.02):
        self.T = T
        self.betas = torch.linspace(beta_start, beta_end, T)
        self.alphas = 1.0 - self.betas
        self.alpha_bar = torch.cumprod(self.alphas, dim=0)

    def to(self, device):
        self.betas = self.betas.to(device)
        self.alphas = self.alphas.to(device)
        self.alpha_bar = self.alpha_bar.to(device)
        return self

    def q_sample(self, x0, t, noise):
        ab = self.alpha_bar[t][:, None, None, None]
        return ab.sqrt() * x0 + (1 - ab).sqrt() * noise

    @torch.no_grad()
    def p_sample(self, model, x_t, t_scalar):
        B, device = x_t.size(0), x_t.device
        t_vec = torch.full((B,), t_scalar, dtype=torch.long,
            device=device)
        beta_t = self.betas[t_scalar]
        alpha_t = self.alphas[t_scalar]
        alpha_bar_t = self.alpha_bar[t_scalar]
        eps_pred = model(x_t, t_vec)
```

```
        coef = beta_t / (1 - alpha_bar_t).sqrt()
        mean = (x_t - coef * eps_pred) / alpha_t.sqrt()
        if t_scalar == 0:
            return mean
        return mean + beta_t.sqrt() * torch.randn_like(x_t)

    @torch.no_grad()
    def sample(self, model, shape, device):
        model.eval()
        x = torch.randn(shape, device=device)
        for t in reversed(range(self.T)):
            x = self.p_sample(model, x, t)
        return x
```

7.6 Energy-Tracked Training Loop (Autoencoder)

```
project_name = f"qpj_{config['model_name']}"
    tracker = create_tracker(project_name, config["log_dir"])

    epoch_logs = []
    tracker.start()
    training_start = time.time()

    for epoch in range(1, config["epochs"] + 1):
        model.train()
        epoch_loss  = 0.0
        epoch_power = []
        epoch_start = time.time()

        pbar = tqdm(train_loader, desc=f"Epoch
           {epoch:03d}/{config['epochs']}", leave=False)
        for images, _ in pbar:
            images = images.to(device)
            recon, _ = model(images)
            loss = criterion(recon, images)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item()
            pw = get_gpu_power_watts()
```

```
            if pw > 0:
                epoch_power.append(pw)
            pbar.set_postfix(loss=f"{loss.item():.4f}", gpu_w=f"{pw:.1f}W")

        scheduler.step()

        # Validation loss
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for images, _ in test_loader:
                images = images.to(device)
                recon, _ = model(images)
                val_loss += criterion(recon, images).item()
        val_loss /= len(test_loader)

        epoch_time  = time.time() - epoch_start
        avg_loss    = epoch_loss / len(train_loader)
        avg_power   = float(np.mean(epoch_power)) if epoch_power else 0.0
        energy_j    = avg_power * epoch_time

        epoch_logs.append({
            "epoch"         : epoch,
            "train_loss"    : round(avg_loss,  6),
            "val_loss"      : round(val_loss,  6),
            "epoch_time_s"  : round(epoch_time, 2),
            "avg_power_w"   : round(avg_power,  2),
            "epoch_energy_j": round(energy_j,   2),
        })

        print(f"  Epoch {epoch:03d} | Train: {avg_loss:.4f} | "
              f"Val: {val_loss:.4f} | {epoch_time:.1f}s | "
              f"{avg_power:.1f}W | {energy_j:.1f}J")

        if epoch % 10 == 0:
            ckpt = os.path.join(config["checkpoint_dir"],
                f"{config['model_name']}_epoch{epoch:03d}.pt")
            save_checkpoint(model, optimizer, epoch, avg_loss, ckpt)
```

```python

            # Generate 16 reconstructions for sample grid
            model.eval()
            sample_imgs, _ = next(iter(test_loader))
            sample_imgs = sample_imgs[:16].to(device)
            with torch.no_grad():
                recon_imgs, _ = model(sample_imgs)
            samp = os.path.join(config["sample_dir"],
                f"{config['model_name']}_recon_epoch{epoch:03d}.png")
            save_sample_grid(recon_imgs, samp,
                             title=f"AE Reconstructions - Epoch {epoch}", n=16, nrow=8, denorm_fn=denormalize)
            print(f"  Checkpoint: {ckpt}")

    emissions          = tracker.stop()
    total_time         = time.time() - training_start

    cc_energy_j = get_total_energy_joules(config["log_dir"], project_name)
    nvml_energy_j = sum(e["epoch_energy_j"] for e in epoch_logs)
    # Prefer CodeCarbon energy if available, otherwise use NVML-based estimate
    total_energy_j = cc_energy_j if cc_energy_j > 0 else nvml_energy_j
```

7.7 FID Computation

```python
from torchmetrics.image.fid import FrechetInceptionDistance

def compute_fid(real_loader, fake_images, device):
    fid = FrechetInceptionDistance(feature=2048).to(device)
    # Feed real images
    for imgs, _ in real_loader:
        fid.update(to_uint8(imgs).to(device), real=True)
    # Feed generated images
    for i in range(0, len(fake_images), batch_size):
        fid.update(to_uint8(fake_images[i:i+batch_size]).to(device), real=False)
    return fid.compute().item()

```

```
# Quality per Joule
qpj = 1.0 / (fid_score * total_energy_j)
```

7.8 WGAN-GP Gradient Penalty

```
# Gradient penalty for WGAN-GP
def gradient_penalty(critic, real, fake, device,
    lambda_gp=10.0):
    B = real.size(0)
    alpha = torch.rand(B, 1, 1, 1,
        device=device).expand_as(real)
    interpolated = (alpha * real + (1 - alpha) *
        fake.detach()).requires_grad_(True)
    c_interp = critic(interpolated)
    gradients = torch.autograd.grad(
        outputs=c_interp, inputs=interpolated,
        grad_outputs=torch.ones_like(c_interp),
        create_graph=True, retain_graph=True,
    )[0]
    gradients = gradients.view(B, -1)
    return lambda_gp * ((gradients.norm(2, dim=1) - 1) **
        2).mean()
```

7.9 DDPM Noise Prediction Training Step

```
pbar = tqdm(train_loader, desc=f"Epoch
    {epoch:03d}/{config['epochs']}", leave=False)
    for x0, _ in pbar:
        x0 = x0.to(device)
        t = torch.randint(0, config["T"], (x0.size(0),),
            device=device)
        noise = torch.randn_like(x0)
        x_t = scheduler_diff.q_sample(x0, t, noise)

        eps_pred = model(x_t, t)
        loss     = F.mse_loss(eps_pred, noise)
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
```

```

        epoch_loss += loss.item()
        pw = get_gpu_power_watts()
        if pw > 0:
            epoch_power.append(pw)
        pbar.set_postfix(loss=f"{loss.item():.4f}",
           gpu_w=f"{pw:.1f}W")
```

References

[1] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[2] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. ICLR*, 2014.

[3] I. Goodfellow et al., "Generative adversarial nets," in *NeurIPS*, 2014.

[4] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016.

[5] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in *NeurIPS*, 2017.

[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *NeurIPS*, 2020.

[7] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *NeurIPS*, 2017.

[8] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE ICCV*, pp. 3730–3738, 2015.

[9] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. ACL*, pp. 3645–3650, 2019.

[10] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[11] CodeCarbon contributors, "CodeCarbon: Track and reduce CO_2 emissions from compute," https://codecarbon.io/, 2020.